

Projektowanie i Analiza Algorytmów	
Kierunek <i>Informatyczne Systemy Automatyki</i>	Termin <i>wtorek 17⁰⁵ – 18⁴⁵</i>
Imię, nazwisko, numer albumu <i>Jan Hulbój 284197</i>	Data <i>16.06.2026</i>
Link do projektu https://github.com/mlody-jano/algorithm_design_and_analysis	



Politechnika
Wrocławska

SPRAWOZDANIE – MINIPROJEKT NR 3

Poniższy dokument jest kompleksowym sprawozdaniem opisującym badania przeprowadzone w ramach trzeciego projektu na przedmiocie **Projektowanie i Analiza Algorytmów**. Przedmiotem owego projektu było zastosowanie **algorytmu minmax()** z cięciami α β , do zaimplementowania dowolnej wybranej gry strategicznej. Implementacja stworzona w ramach tego projektu pozwala graczowi na definicję rozmiaru planszy oraz poziomu głębokości przeszukiwania algorytmu, pozwalając na szybkie i efektywne dostosowywanie działania algorytmu. Gra została rozszerzona również o **interfejs graficzny** utworzony przy pomocy biblioteki SFML, który odpowiada za graficzne API pomiędzy graczem a programem.

Spis treści

1 Wstęp teoretyczny	1
1.1 Gra w kółko i krzyżyk	1
1.2 Algorytm minmax()	1
1.3 Alfa-beta pruning	2
1.4 Ewaluacja stanu planszy	2
2 Wnioski	2

1 Wstęp teoretyczny

1.1 Gra w kółko i krzyżyk

Gra w kółko i krzyżyk (*ang. Tic Tac Toe*) jest grą strategiczną polegającą na ułożeniu przez gracza na planszy rozmiarów $n \times n$ ciągu n tych samych, tj. swoich symboli. Dozwolone linie wygrywające zawierają się w liniach wertykalnych, horyzontalnych lub diagonalnych wewnątrz pola planszy. Dodatkowo, pola objęte przez jednego z graczy podczas swojego ruchu nie podlegają zmianie podczas gry.

1.2 Algorytm minmax()

Algorytm minmax() jest jednym z algorytmów do minimalizowania maksymalnych możliwych strat w grze strategicznej. Bazuje on na przeszukiwaniu drzewa stanów planszy i obecność graczy **min** i **max**, odpowiednio minimalizujących i maksymalizujących obecny wynik. Algorytm na podstawie heurystyki ocenia wartość danego ustawienia i wybiera optymalną trasę. Wynikiem tego algorytmu jest optymalny ruch w zakresie zadanej głębokości, prowadzący do najlepszego stanu końcowego jaki został zwrócony przez funkcję ewaluacyjną.

Algorytm operuje w złożoności obliczeniowej $O(b^d)$, gdzie b stanowi **współczynnik rozgałęzienia**, natomiast d odpowiada **maksymalnej głębokości** przeszukiwania algorytmu. Teoretycznie można więc założyć, że wraz ze zwiększaniem aktualnej głębokości algorytmu, czas wymagany na jego obliczenie powinien spadać, ponieważ obecnie przeszukiwane drzewo ma niższy współczynnik rozgałęzienia.

Odpowiedni dobór głębokości przeszukiwania jest kluczowym czynnikiem do poprawnego działania algorytmu, ponieważ zbyt duży próg przeszukiwania może prowadzić do lepszych efektów pod względem optymalności globalnej, lecz nieakceptowalnie dużych czasów wykonania pojedynczego ruchu przez algorytm. Natomiast z drugiej strony, zbyt niski próg przeszukiwania może skutkować tym, że algorytm staje się zachłanny i wybiera ścieżkę prowadzącą do lokalnie optymalnej opcji, nie zwracając uwagi na dalszy przebieg gry, ale wykonując to w bardzo szybkim czasie.

1.3 Alfa-beta pruning

Alfa-beta pruning jest jedną z optymalizacji algorytmu `minimax()`, która pozwala znacząco zmniejszyć liczbę przeszukiwanych węzłów w wybranych poddrzewach, jednocześnie pozwalając na zwiększenie głębokości przeszukiwania, nie zwiększając przy tym narztu czasowego.

Opiera się on na wartościach α β obliczanych podczas przechodzenia przez drzewo stanów. W przypadku znalezienia większej wartości α niż obecnie zapisana przez gracza maksymalizującego, nadpisuje wartość, analogicznie z wartością β dla gracza minimalizującego. Odcięcie gałęzi następuje w momencie gdy znajdziemy takie wartości że $\alpha \geq \beta$, co jest dowodem na to, że w danej gałęzi gracz maksymalizujący nie znajdzie już większej wartości niż wcześniej zagwarantowana α , gdy gracz minimalizujący nie zmieni strategii wyboru.

To usprawnienie znacząco zmniejsza złożoność algorytmu `minimax()`, redukując ją do $O(2b^{\frac{d}{2}})$. Pozwala to na przeszukiwanie ruchów około dwa razy głębiej, bez tworzenia dodatkowego narztu czasowego.

1.4 Ewaluacja stanu planszy

Ewaluacja stanu planszy w grze Kółko i Krzyżyk została wykonana przez funkcję heurystyczną **S**, która ocenia stan planszy przez sumowanie stanów każdej z możliwych linii przy użyciu funkcji $f(x, o)$ na planszy na podstawie danego wzoru:

$$f(x, o) = \begin{cases} 10^x & x > 0 \wedge o = 0 \\ -10^o & x = 0 \wedge o > 0 \\ 0 & x > 0 \wedge o > 0 \end{cases} \quad (1)$$

$$S = \sum_{i=1}^{2n+2} f(x, o) \quad (2)$$

, gdzie **S** odpowiada wynikowi planszy, natomiast **k** odpowiada ilości symbolów gracza w linii, a **n** odpowiada rozmiarowi planszy.

Funkcja sprawdza kolejno każdą z linii, i ocenia jej wynik, a następnie dodaje go do sumy. Jeżeli w jednej linii znajdują się dwa różne symbole, wynik tej linii od razu jest zerowany, ponieważ nie prowadzi do wygranej żadnego z graczy.

2 Wnioski

Zaimplementowanie algorytmu `minimax()` pozwoliło na jasne i przejrzyste zobrazowanie działania SI w kontekście optymalnego wyboru ruchu w grze Kółko i Krzyżyk. Proces przeszukiwania stanów planszy przez rekurencyjne wywoływanie algorytmu i ewaluację planszy przy pomocy heurystyki pozwoliły łatwo zrozumieć metodologię dojścia do najlepszego ruchu SI, a także istotność odpowiedniego doboru głębokości przeszukiwania.

Cięcia alfa-beta okazały się być znaczącym usprawnieniem w kontekście złożoności algorytmu i ilości odwiedzanych węzłów drzewa. Stosując powyższą technikę możliwa jest redukcja przeszukiwanych węzłów o około połowę, pozwalając na efektywne przeszukiwanie drzewa na dwa razy większej głębokości, bez zmiany narztu czasowego.

Bibliografia

1. *Alpha-Beta Pruning in Adversarial Search Algorithms* **online**GeeksforGeeks. [dostęp: 2026-06-16]. Dostępne z: <https://www.geeksforgeeks.org/artificial-intelligence/alpha-beta-pruning-in-adversarial-search-algorithms/>.

2. Minimax. W: *Wikipedia* **online**[b.d.] [dostęp: 2026-06-16]. Dostępne z: <https://en.wikipedia.org/w/index.php?title=Minimax&oldid=1353211022>.
3. *Tutorials - Simple and Fast Multimedia Library* **online**[dostęp: 2026-06-16]. Dostępne z: <https://www.sfml-dev.org/tutorials/3.1/>.